



معماری کامپیوتر

بهار ۱۴۰۵

استاد: دکتر سربازی آزاد

محسن صلاح - ۴۰۳۱۰۶۲۳۸

تمرین شماره ۷

۱ مقدمه

هدف از این تمرین، بهبود پردازنده‌ی تک‌چرخه‌ای طراحی‌شده در تمرین ۵ و تبدیل کردن آن به یک پردازنده‌ی خط لوله‌ی (Pipeline) پنج مرحله‌ای است. در معماری خط لوله، چندین دستور به‌طور هم‌زمان در مراحل مختلف پردازش قرار می‌گیرند و توان عملیاتی سیستم به‌طور چشم‌گیری افزایش می‌یابد. همچنین این هم‌پوشانی پدیده‌ی Hazards را ایجاد می‌کند که در این تمرین با دو روش ارسال رو به جلو (Forwarding) و توقف (Stall) برطرف می‌شوند.

پنج زیرماژول تمرین پنجم (واحد کنترل (control_unit)، بانک ثبات (regfile)، واحد حساب و منطق (alu)، ضرب‌کننده (multiplier) و تقسیم‌کننده (divider)) بدون هیچ تغییری استفاده شدند. تمام منطق جدید خط لوله، شامل ثبات‌های واسط، تشخیص و رفع مخاطرات داده و کنترلی، در ماژول سطح بالا (main) پیاده‌سازی شده است.

۲ مراحل خط لوله و ثبات‌های واسط

پردازنده از پنج مرحله‌ی پشت سر هم تشکیل شده است: واکنشی دستور (Fetch, F)، رمزگشایی (Decode)، اجرا (Execute, E)، دسترسی به حافظه (Memory, M) و بازنویسی (Write Back, W). در هر لبه‌ی بالارونده‌ی ساعت، محتوای چهار ثبات واسط (FD, DE, EM, MW) به‌روزرسانی می‌شود. هر ثبات واسط یک بیت اعتبار (Valid) دارد که در صورت صفر بودن، دستور موجود در آن مرحله به‌عنوان NOP در نظر گرفته می‌شود. این بیت برای تمام ثبات‌ها هنگام Reset صفر می‌شود. فیلدهای ذخیره‌شده در هر ثبات واسط در این جدول آمده است:

جدول ۱: ثبات‌های واسط خط لوله و فیلدهای آن‌ها

ثبات واسط	فیلدهای ذخیره‌شده
FD	Instr, PC, Valid
DE	(sign extended) Imm, RtVal, RsVal, Instr, PC, Valid, Branch, MemWrite, MemRead, RegWrite, WriteReg, Jal, ALUSrc, ALUControl, MemtoReg
EM	RegWrite, WriteReg, RtVal, ALUResult, Instr, PC, Valid, Jal, MemtoReg, MemWrite, MemRead
MW	RegWrite, WriteReg, ReadData, ALUResult, PC, Valid, Jal, MemtoReg

ثبات شمارنده‌ی برنامه (PC) به صورت Word Index است، یعنی در هر چرخه‌ی عادی یک واحد افزایش می‌یابد ($pc_plus_1 = PC + 1$) و مستقیماً به خروجی InstrAddr متصل شده. آدرس داده نیز از رابطه‌ی $DataAddr = EM_ALUResult \gg 2$ به دست می‌آید (چون رجیسترها آدرس‌های بایتی نگه می‌دارند). منطق تعیین مقدار بعدی PC نیز با یک مالتی‌پلکسر چهارحالتی پیاده‌سازی شده است:

```

1 wire [31:0] next_pc = branch_taken ? branch_target :
2   jump_in_D ? jump_target :
3   stall ? PC :
4   pc_plus_1;

```

پرش شرطی فعال (branch_taken) بالاترین اولویت را دارد، پس از آن پرش غیرشرطی (jump_in_D)، و بعد از آن توقف خط لوله (stall) که PC را ثابت نگه می‌دارد، و در غیر این صورت افزایش عادی pc_plus_1 است. در ادامه نحوه‌ی محاسبه‌ی این سیگنال‌ها و سازوکار کلی رفع مخاطرات توضیح داده می‌شود.

۳ مدیریت مخاطرات داده

۱.۳ تشخیص مخاطره و مکانیزم توقف

اولین گام برای تشخیص Hazard این است که مشخص کنیم آیا دستور در حال رمزگشایی (مرحله‌ی D) به ثبات‌های rs یا rt نیاز دارد یا خیر. برای این منظور دو سیگنال ترکیبی $uses_rs$ و $uses_rt$ بر اساس opcode و funct دستور جاری در ثبات FD تعریف شده‌اند:

```

1 wire uses_rs = FD_Valid && (
2   (FD_opcode == RTYPE && (FD_funcnt == F_ADD || FD_funcnt == F_SUB ||
3     FD_funcnt == F_MUL || FD_funcnt == F_DIV || FD_funcnt ==
4     F_JR)) ||
5   (FD_opcode == ADDI) || (FD_opcode == SUBI) ||
6   (FD_opcode == LW) || (FD_opcode == SW) ||
7   (FD_opcode == BEQ));
8 wire uses_rt = FD_Valid && (
9   (FD_opcode == RTYPE && (FD_funcnt == F_ADD || FD_funcnt == F_SUB ||
10     FD_funcnt == F_MUL || FD_funcnt == F_DIV)) ||
11   (FD_opcode == SW) || (FD_opcode == BEQ));

```

در گام بعد، با بررسی ثبات‌های واسطه DE و EM، مشخص می‌شود که آیا دستور قرار گرفته در مرحله‌ی E یا M قرار است در ثباتی بنویسد که دستور D به آن وابسته است یا خیر:

```

1 wire e_will_write = DE_Valid && DE_RegWrite && DE_WriteReg != 5'd0;
2 wire m_will_write = EM_Valid && EM_RegWrite && EM_WriteReg != 5'd0;
3
4 assign data_hazard = FD_Valid && (
5   (uses_rs && e_will_write && DE_WriteReg == FD_rs) ||
6   (uses_rs && m_will_write && EM_WriteReg == FD_rs) ||
7   (uses_rt && e_will_write && DE_WriteReg == FD_rt) ||
8   (uses_rt && m_will_write && EM_WriteReg == FD_rt));

```

هرگاه data_hazard فعال شود، سیگنال stall بالا می‌رود و دو اتفاق به صورت موازی رخ می‌دهند: ثابت Freeze، FD می‌شود تا دستور مخاطره‌دار در مرحله‌ی D بماند، و در همان حال یک Bubble با صفر کردن DE_Valid به مرحله‌ی E تزریق می‌شود. نکته‌ی مهم این است که ثابت DE به‌جای منجمد شدن، حباب دریافت می‌کند. اگر DE منجمد می‌شد، دستور مخاطره‌ساز برای چرخه‌های بعد همچنان در مرحله‌ی E باقی می‌ماند، سیگنال e_will_write مدام فعال می‌ماند و data_hazard هرگز صفر نمی‌شد و یک توقف بی‌نهایت رخ می‌داد. با تزریق حباب، دستور مخاطره‌ساز به‌طور طبیعی به مراحل بعدی پیش می‌رود و سیگنال‌های e_will_write و m_will_write در طول چرخه‌های بعدی صفر می‌شوند.

تعداد چرخه‌های توقف به جایگاه دستور تولیدکننده بستگی دارد: اگر آن دستور در مرحله‌ی E باشد، دو چرخه توقف لازم است تا نتیجه‌اش به مرحله‌ی W برسد و بتواند در دسترس قرار گیرد. اگر در مرحله‌ی M باشد، یک چرخه کافی است.

۲.۳ ارسال رو به جلو از مرحله‌ی W به D

برای کاهش تعداد توقف‌های لازم، یک مسیر ارسال رو به جلو از مرحله‌ی W به مرحله‌ی D پیاده‌سازی شده است. اگر دستوری که در حال بازنویسی در مرحله‌ی W است (ثبات MW) در همان ثباتی بنویسد که دستور در D به آن نیاز دارد، مقدار محاسبه‌شده مستقیماً ارسال می‌شود و نیازی به خواندن از بانک ثبات نیست:

```

1 wire [31:0] d_rs_val =
2   (MW_Valid && MW_RegWrite && MW_WriteReg == FD_rs && FD_rs != 5'd0)
3   ? MW_WriteData : reg_rs_data;
4 wire [31:0] d_rt_val =
5   (MW_Valid && MW_RegWrite && MW_WriteReg == FD_rt && FD_rt != 5'd0)
6   ? MW_WriteData : reg_rt_data;
```

مقدار MW_WriteData به‌صورت ترکیبی محاسبه می‌شود: برای JAL آدرس بازگشت، برای LW داده‌ی خوانده‌شده از حافظه، و برای سایر دستورها نتیجه‌ی ALU. اگرچه بانک ثبات سنکرون است و در همان لبه‌ای که MW_Valid بالا می‌رود می‌نویسد، همچنان به این مسیر نیاز داریم چون خواندن ترکیبی بانک ثبات هنوز مقدار قدیمی پیش از آن لبه را می‌بیند.

نکته‌ی مهم: بانک ثبات در این پیاده‌سازی از روی سیگنال‌های ثابت EM (نه MW) نوشته می‌شود:

```

1 wire [31:0] em_write_data =
2   EM_Jal ? ((EM_PC + 32'd1) << 2) :
3   (EM_MemtoReg==2'b01)? DataIn :
4   EM_ALUResult;
5
6 regfile rf(.RegWrite(EM_RegWrite && EM_Valid),
7   .WriteReg(EM_WriteReg),
8   .WriteData(em_write_data), ...);
```

دلیل این است که تست‌بنچ بلافاصله پس از فعال‌شدن W_Valid مقادیر ثبات‌ها را بررسی می‌کند. چون نوشتن بانک ثبات و لچ شدن EM به MW هر دو Non-blocking هستند و روی همان لبه‌ی ساعت رخ می‌دهند، در لحظه‌ی $\text{posedge} + \varepsilon$ هر دو تکمیل شده‌اند. اگر بازنویسی از روی سیگنال‌های MW انجام می‌شد، یک چرخه دیرتر از MW_Valid رخ می‌داد و تمام دستورات در تست‌بنچ شکست می‌خوردند.

در نهایت، مقادیر d_rs_val و d_rt_val هنگام پیشروی دستور از D به E در ثبات‌های DE_RsVal و DE_RtVal ذخیره می‌شوند و در مرحله‌ی E توسط ALU و ماژول‌های ضرب/تقسیم استفاده می‌گردند.

۳.۳ توقف برای دستورات MUL و DIV

ماژول‌های ضرب و تقسیم هر کدام به ۳۲ کلاک نیاز دارند. در این مدت سیگنال mul_div_stall فعال می‌ماند تا این کارها انجام شود: ثبات FD منجمد می‌شود تا دستورات جدید وارد خط لوله نشوند، و ثبات DE نیز منجمد می‌شود تا عملوندهای DIV/MUL در طول محاسبه بدون تغییر باقی بمانند. برخلاف توقف‌های مخاطره‌ی داده، در اینجا منجمد کردن DE کاملاً صحیح است، چون دستور DIV/MUL خودش باید در مرحله‌ی E بماند تا محاسبه به اتمام برسد، نه اینکه یک دستور دیگر جایش را بگیرد. این توقف با سیگنال mul_div_stall از سیگنال data_hazard جدا نگه داشته شده تا در بلوک به‌روزرسانی ثبات‌ها بتوان رفتار متفاوتی برای هر کدام تعریف کرد.

۴ مدیریت مخاطرات کنترلی

۱.۴ پرش‌های غیرشرطی در مرحله‌ی D

دستورات J، JAL و JR در مرحله‌ی D شناسایی می‌شوند، چون مقصد پرش آن‌ها مستقیماً از فیلدهای دستور یا ثبات‌های رمزگشایی‌شده قابل محاسبه است و نیازی به واحد ALU ندارد:

```
1 assign jump_in_D = FD_Valid && !stall &&
2   ((FD_opcode == J) || (FD_opcode == JAL) ||
3   (FD_opcode == RTYPE && FD_func == F_JR));
4
5 assign branch_taken = DE_Valid && DE_Branch && zero && !mul_div_stall;
6
7 wire flush_FD = jump_in_D || branch_taken;
```

شرط stall! در تعریف jump_in_D برای دستور JR اهمیت ویژه‌ای دارد: اگر ثباتی که JR به آن پرش می‌کند هنوز در راه باشد (مخاطره‌ی داده)، ابتدا باید توقف رخ دهد تا مقدار صحیح در d_rs_val آماده شود، و تنها بعد از آن jump_in_D فعال گردد.

مقصد پرش بسته به نوع دستور متفاوت است. برای J و JAL، آدرس ۲۶ بیتی فیلد دستور مستقیماً به‌عنوان شاخص کلمه‌ای استفاده می‌شود (jump_target = {6'b0, FD_jaddr}). برای JR، مقدار ثبات که یک آدرس بایتی است به شاخص کلمه‌ای تبدیل می‌شود (jump_target = d_rs_val >> 2).

هر پرش غیرشرطی دستوری را که بلافاصله پس از آن fetch شده باطل می‌کند. این کار با فعال‌شدن flush_FD انجام می‌شود که فقط FD_Valid را صفر می‌کند. خود دستور پرش (مثلاً JAL) بدون هیچ تغییری از D به E و سپس به W پیشروی می‌کند تا بازنویسی در ثبات در مرحله‌ی W انجام شود. از آنجا که JAL باید آدرس بایتی دستور بعد از خودش را در \$31 ذخیره کند و PC شاخص کلمه‌ای است، مقدار ذخیره‌شده برابر (EM_PC + 1) << 2 است (همان em_write_data در حالت Jal=1)؛ و ثبات مقصد در مرحله‌ی D صریحاً برابر \$31 تنظیم می‌شود:

$$DE_WriteReg = Jal ? 31 : (RegDst ? FD_rd : FD_rt)$$

۲.۴ پرش شرطی (BEQ) در مرحله‌ی E

دستور BEQ باید در مرحله‌ی E شناسایی شود، چون بررسی برابری دو ثبات به سیگنال Zero از خروجی واحد ALU نیاز دارد که تنها در این مرحله در دسترس است. به همین دلیل، مخاطره‌ی کنترلی آن یک مرحله دیرتر از پرش‌های غیرشرطی تشخیص داده می‌شود. هنگامی که branch_taken فعال می‌شود، دو اقدام موازی انجام می‌گیرد: flush_FD فعال می‌شود تا دستور که در آن لحظه در مرحله‌ی D است (و از مسیر اشتباه آمده) باطل شود، و DE_Valid نیز در همان چرخه صفر می‌شود تا یک حباب به مرحله‌ی E تزریق گردد.

آدرس هدف پرش در مرحله‌ی E از رابطه‌ی زیر محاسبه می‌شود:

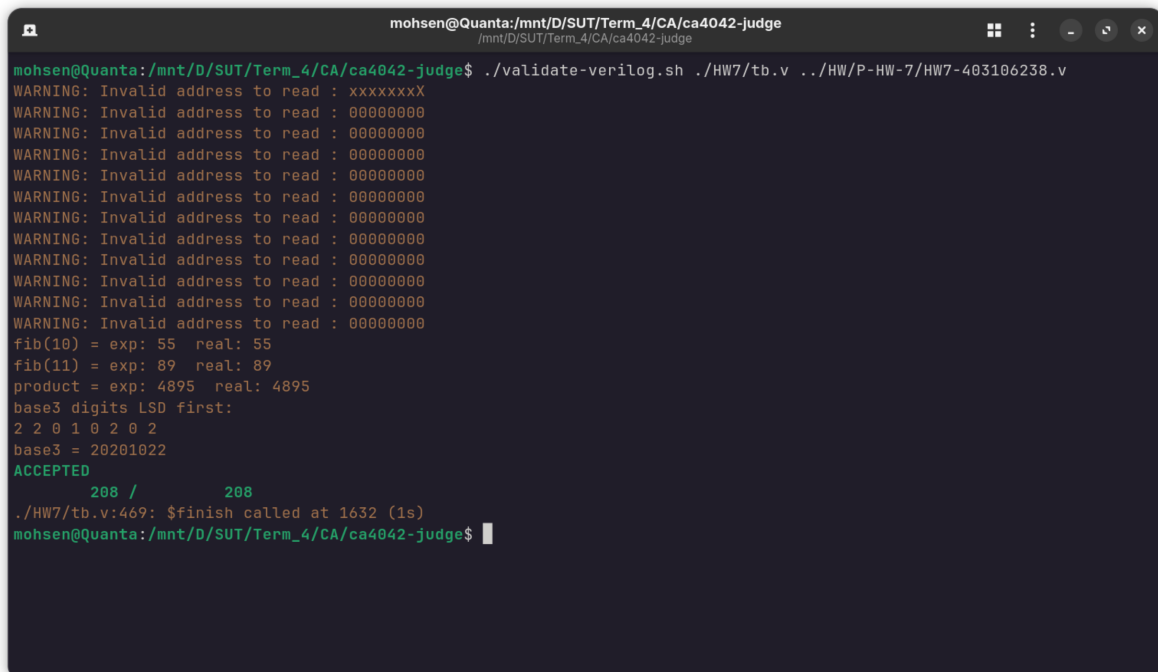
$$\text{branch_target} = (\text{DE_PC} + 1) + \text{DE_Imm}$$

که در آن DE_PC شاخص کلمه‌ای دستور BEQ است و DE_Imm مقدار sign-extend_offset شده است که مستقیماً به صورت تعداد کلمه استفاده می‌شود.

نکته‌ی طراحی مهم این است که خود دستور BEQ نباید در مرحله‌ی E یا M باطل شود. تست‌بنچ برای تأیید اجرای هر دستور منتظر می‌ماند تا W_Valid فعال و W_PC برابر آدرس آن دستور شود؛ بنابراین BEQ باید بدون هیچ تغییری تا مرحله‌ی W پیش برود، هرچند که هیچ بازنویسی‌ای در ثبات‌ها ندارد.

۵ اجرای آزمون

این کد را با تست‌بنچ داده شده بررسی کردیم و نتیجه‌ی آن به این صورت می‌باشد:



```
mohsen@Quanta:/mnt/D/SUT/Term_4/CA/ca4042-judge$ ./validate-verilog.sh ./HW7/tb.v ../HW/P-HW-7/HW7-403106238.v
WARNING: Invalid address to read : xxxxxxxX
WARNING: Invalid address to read : 00000000
WARNING: Invalid address to read : 00000000
WARNING: Invalid address to read : 00000000
WARNING: Invalid address to read : 00000000
WARNING: Invalid address to read : 00000000
WARNING: Invalid address to read : 00000000
WARNING: Invalid address to read : 00000000
WARNING: Invalid address to read : 00000000
WARNING: Invalid address to read : 00000000
WARNING: Invalid address to read : 00000000
WARNING: Invalid address to read : 00000000
fib(10) = exp: 55 real: 55
fib(11) = exp: 89 real: 89
product = exp: 4895 real: 4895
base3 digits LSD first:
2 2 0 1 0 2 0 2
base3 = 20201022
ACCEPTED
208 / 208
./HW7/tb.v:469: $finish called at 1632 (1s)
mohsen@Quanta:/mnt/D/SUT/Term_4/CA/ca4042-judge$
```

شکل ۱: اجرای تست‌بنچ تمرین